MemeScreen

Phase 2 addendum to the app schematic: the MemeScreen trading wallet with server-side execution and 24/7 automatic exits.

Document date	October 29, 2026
Build state	Web app build 2026-10-29a, phone shell 11, execution service memescreen/exec 1.0.0. Everything below is built and tested end to end against the real Jupiter API and a real Back4App login, with the on-chain send simulated (FAKE_CHAIN). The first real-money run happens once the service is deployed and the wallet is funded.
What changed for the user	A third kind of account next to paper and Phantom: the MemeScreen wallet. No Phantom approval per order. Stop-loss and take-profit sell by themselves, with the phone in a pocket.
Rule	The user's key is held by the MemeScreen execution service, encrypted. The web app never sees it. Withdrawals need the account password again and are capped per day.

1. The three accounts

Account	Who signs	Exits	Money
Paper	Nobody (simulation)	Instant, simulated	$10,000 of play money
Phantom (Phase 1)	The user, in Phantom, every order	Alert + one tap, only while the app is open	The user's own Phantom wallet
MemeScreen wallet (Phase 2)	The execution service, with the user's server-held key	Automatic, 24/7, on the server	A wallet the user funds from Phantom and withdraws back to

2. Architecture

The execution service ("ms-exec") is a Node service with Postgres (SQLite when run locally). It sits between the app and the chain and is the only thing that ever holds a user key. Back4App keeps doing what it did: users, sessions, paper accounts, competitions, push devices. The service verifies the app's Back4App session token on every call, and pushes notifications through the existing pushToUser cloud function using the master key.

Piece	Where	Talks to
Web + phone app (src/twallet.js)	GitHub Pages / Expo Go	ms-exec with the session token; Back4App as before; Jupiter for the preview quote
ms-exec (memescreen/exec)	Railway or Render + Postgres	Back4App (/users/me, /login, pushToUser), Jupiter (quote, swap), Solana RPC (send, confirm, balances), DexScreener (prices)
Watcher (inside ms-exec)	Same process, every 3 s	Postgres (armed positions), DexScreener prices, placeOrder()

2.1 Data (Postgres / SQLite)

Table	What it holds
wallets	user_id, pubkey, enc_secret + iv (AES-256-GCM under MASTER_KEY), withdraw_to, daily_limit_sol
orders	every attempt: side, size, quote, tx_sig, status (quoting, sent, confirmed, failed, expired), error, source (user / watcher_sl / watcher_tp)
positions	qty, cost_usd, sol_in, entry_px, tp_px, sl_px, trail_pct, high_px, armed, hit
withdrawals	to, sol, tx_sig, status, time (feeds the daily limit)
events	audit trail: wallet_created, order_confirmed, exit_hit, exit_sold, exit_failed, withdraw_confirmed

2.2 API (Authorization: Bearer <Back4App session token>)

Route	Body	Returns
POST /v1/wallet	—	{ pubkey } — creates once, returns the same after
GET /v1/wallet	—	{ pubkey, sol, solUsd, spendable, tokens[] }
POST /v1/order	{ mint, side, solIn | pct, slippageBps, tp, sl, trailPct }	{ orderId, sig, status, out } after on-chain confirmation
POST /v1/exits	{ mint, tp, sl, trailPct } (% of entry; null clears)	{ ok, position }
GET /v1/positions	—	positions with price, value, pnl_usd, pnl_pct, armed, hit
GET /v1/orders	—	last 100 orders
POST /v1/withdraw	{ to, sol, password }	{ sig } — password re-checked, ■1 per day
GET /healthz	—	watcher ticks, last check, fired, errors

2.3 Order path

1. Verify the session token with Back4App /users/me (cached 60 s).
2. Size: buy = SOL in (pre-check balance ≥ size + ■0.012 reserve); sell = percent of the on-chain token balance.
3. Jupiter /quote → /swap (wrap/unwrap SOL, dynamic slippage, priority fee cap ■0.002).
4. Decrypt the key inside the request, sign the versioned transaction, wipe the key.
5. sendRawTransaction, then poll getSignatureStatuses up to 60 s. Expired blockhash: re-quote and resend once.
6. Record the order; update the position (entry = SOL spent × SOL price ÷ tokens received); arm TP/SL/trailing; push "Bought …".

2.4 Watcher

1. Every 3 s: load armed positions that have an exit; price them in one DexScreener call per 30 mints.
2. Trailing stop ratchets sl_px up with each new high.
3. On a crossed level: an exclusive UPDATE sets armed = 0 (an exit fires once, even with two instances), then sell 100% through the same placeOrder.
4. Push "Stop-loss hit at $…: sold … for ■…". On failure: 3 attempts, then re-arm with a 60 s cooldown and push "sale failed, retrying".

2.5 Key custody

- MASTER_KEY (32 random bytes) lives only in the hosting provider's environment variables. Losing it loses every wallet; the owner keeps one offline copy.
- Keys are never cached, never logged (a log filter refuses 64-byte base58 strings), never returned by any route.
- Withdrawals: the account password is checked again with Back4App /login (never stored), the destination is remembered, ■1 per 24 h by default, 5 per minute.
- Orders: 10 per minute per user. CORS restricted to the site origin in production (ALLOWED_ORIGINS).
- Before public users: move MASTER_KEY to a cloud KMS (one function in keys.js) and get the custody/legal review from the team plan.

3. In the app

Where	What is new
Broker window	A "MemeScreen wallet" card. First tap creates the wallet on the server and opens the funding screen (address, QR as a Solana Pay link Phantom can scan, "Open in wallet app", copy).
Header account switcher	Third option: "MemeScreen wallet · auto". The header shows ■ balance, "auto · N tokens", and free SOL.
Trade ticket	Badge AUTO. Buy: SOL presets and % of spendable, take profit / stop loss in % of entry ("watched by the MemeScreen server every 3 seconds, 24/7"), no "Send for real" checkbox, button "Buy now · executes on the server". Sell: % of holding, "Sell now". Status line: Confirmed + Solscan link.
Chart	AUTO entry line in brand green, Live TP and Live SL lines, straight from the server's position.
Positions tab	Holdings plus P&L; vs entry, exits with an "auto" tag (or "fired SL/TP"), an Edit button that changes TP/SL on the server, Sell.
+ Deposit / Withdraw	In MemeScreen-wallet mode: Deposit opens the funding screen; Withdraw opens a form (destination defaults to the connected Phantom address, amount, password).
Settings → Trading wallet (auto)	Service URL field with "Save & check" (shows online / watcher on / last check), wallet state, "Open wallet".
Phone shell 11	Opens solana: links in the wallet app (Solana Pay) for funding.

4. QA: what was tested

Test	How	Result
Key custody	Unit: encrypt → decrypt round trip, wrong master key fails, secret absent from any serialized row	Pass
Session verification	Unit with a mocked Back4App: good token → user (cached), bad or missing → 401	Pass
Watcher	Unit: price above SL → nothing; below → sells exactly once and disarms; failing seller → 3 attempts then re-arm; trailing stop ratchets	Pass
Order path	Unit with the real Jupiter API: a fresh server wallet gets a real quote and swap transaction, signs it (signature verified with tweetnacl), send mocked; TP/SL recorded at +50% / –25%	Pass
HTTP routes	Boot the service: 401 without token, wallet create idempotent, balance, order validation, exits without position, withdraw (wrong password refused, invalid address refused, transfer built and sent with a mocked RPC, daily limit enforced), healthz	Pass (12 of 12)
End to end in the browser	Real Back4App login, real Jupiter quote, server-side buy of a live token with TP+SL (send simulated), position on the chart and in the Positions tab, Edit exits changed TP/SL on the server, stop-loss forced on the server → the watcher sold the position by itself and the row disappeared, withdraw refused with a wrong password and sent with the right one	Pass
Real money	A deployed service, a funded wallet, a ■0.01 buy and an automatic sell	Not yet — needs the Railway deploy and ■0.05 in the wallet

5. Deploy (owner, about five minutes)

1. railway.app → New Project → Deploy from GitHub repo → ssm-bit/MemeScreen → Settings → Root Directory = memescreen/exec.
2. + New → Database → PostgreSQL (DATABASE_URL is injected into the service).
3. Variables: MASTER_KEY (openssl rand -hex 32; keep an offline copy), BACK4APP_APP_ID, BACK4APP_JS_KEY, BACK4APP_MASTER_KEY, ALLOWED_ORIGINS=https://ssm-bit.github.io, optional RPC_URL (Helius).
4. Settings → Networking → Generate Domain → open https://…/healthz → ok: true.
5. Paste the URL into Settings → Trading wallet in the app (or execUrl in src/config.js and push). Broker → MemeScreen wallet → Fund ■0.05 from Phantom → buy ■0.01 with a 5% stop-loss → wait for the push.